

Informe del CAGS – Project ARES

Español

Nombre del Proyecto **DroneOps Backend**

Objetivo

Desarrollar el backend de una plataforma que permite gestionar y monitorear operaciones de drones para aplicaciones críticas (como reparación en entornos remotos o espaciales). El sistema gestiona telemetría, eventos/alarmas, reportes de daño, reparaciones, inventario de repuestos, auditoría y control de acceso (usuarios con roles: admin, ingeniero, operador).

Funcionalidades Principales

Usuarios y Roles

Roles disponibles: admin, ingeniero, operador.

Permisos:

- **Admin:** acceso completo a todos los módulos.
- **Ingeniero:** puede modificar módulos técnicos (drones, reparaciones, eventos, inventario), pero no gestionar usuarios.
- **Operador:** solo lectura en todos los módulos.

Módulo Drones / Telemetría / Daños

- Registra y consulta datos de telemetría: número de serie, estado de comunicación, carga, versión del software, tiempo en uso, etc.
- Permite reportar daños: tipo, gravedad, condición, foto, ubicación, identificador único.

Módulo Reparaciones

- Registra reparaciones sobre daños detectados.
- Guarda materiales usados (formato JSON), estado, fotos/videos, ubicación, métricas de éxito y hash de seguridad.
- Registro automático de fecha y hora.

Módulo Auditoría (Logs Críticos)

- Registra eventos críticos relacionados con reparaciones (fallos residuales o inconsistencias).
- Cada log incluye descripción, nivel de severidad y fecha.

Módulo Inventario

- Gestiona herramientas y repuestos: nombre, cantidad, categoría, ubicación y hash. • Ingenieros pueden agregar/modificar; operadores solo visualizan.

Módulo Eventos / Alarmas

- Registra eventos generados por drones: alarmas, advertencias o notificaciones. • Cada evento tiene nombre, descripción, tipo, dron asociado, estado y fecha.

API REST + Autenticación JWT

- Implementado con **Django REST Framework** y **Simple JWT**.
- Control de acceso mediante clase personalizada `RolePermission`.

- Endpoints organizados por módulos: drones, reparaciones, auditoría, inventario, eventos y usuarios.

Capa Web / Dashboard (CMS)

- Vistas con **Django Templates**: login, gestión de usuarios y panel principal. • Decoradores personalizados controlan el acceso según rol.

Arquitectura y Configuración Técnica

- **Modelo de Usuario**: extiende AbstractUser con campo role.
- **Permisos personalizados**: clase RolePermission.
- **Base de datos**: SQLite (dev) y PostgreSQL (producción).
- **Hashers**: Argon2 principal, PBKDF2 y BCrypt de respaldo.
- **Seguridad**:
 - DEBUG = False en producción.
 - Cookies seguras y HTTPS activado.
 - ALLOWED_HOSTS configurado.
- **Datos de prueba**: fixtures JSON con telemetría, reparaciones, auditoría, inventario y eventos.
- **Enrutamiento**: routers DRF y rutas Django tradicionales.

Estado Actual y Recomendaciones

- Backend completamente definido (modelos, vistas, serializers y permisos). • Funcionalidades operativas y listas para el frontend.
- **Recomendación**: implementar documentación **Swagger / OpenAPI**.

Project Report – English

Project Name

DroneOps Backend

Goal

Develop the backend of a platform to manage and monitor drone operations in critical environments (remote or space repairs).

The system handles telemetry, alerts, damage reports, repairs, spare parts, auditing, and user access control (admin, engineer, operator).

Key Features

Users and Roles

- **Admin**: full access to all modules.
- **Engineer**: can modify technical modules but cannot manage users. •
- Operator**: read-only access.

Drones / Telemetry / Damage Module

- Records telemetry data: serial number, communication, battery, software version, uptime.
- Manages damage reports: type, severity, condition, image, location, unique ID.

Repairs Module

- Logs repairs, materials (JSON), status, media, location, metrics, and security hash. • Automatic timestamping.

Audit Module (Critical Logs)

- Tracks critical repair-related events (failures, anomalies). • Includes description, severity, and timestamp.

Inventory Module

- Manages tools and parts: name, quantity, category, location, hash. • Engineers edit; operators read-only.

Events / Alerts Module

- Handles drone-generated alerts and warnings.
- Each includes name, description, type, drone, status, timestamp.

REST API + JWT Authentication

- Built with **Django REST Framework** and **Simple JWT**. • Custom RolePermission class for role-based access. • Organized endpoints for all modules.

Web Layer / Dashboard (CMS)

- Django Template-based login, user management, and dashboard. • Access decorators enforce user roles.

Technical Architecture & Configuration

- **User Model:** extends AbstractUser with role.
- **Database:** SQLite (dev), PostgreSQL (prod).
- **Password Hashers:** Argon2 (main), PBKDF2, BCrypt. •
- Security:**
 - DEBUG = False
 - Secure cookies, HTTPS, configured hosts.
- **Test Data:** JSON fixtures for all modules.
- **Routing:** DRF routers + Django views.

Current Status & Recommendations

- Backend fully implemented and operational.
- Ready for frontend integration.
- Recommended: **Swagger / OpenAPI** documentation.

Link del Back: <https://github.com/AngelScarpetta2004/back-end-droneops>